

Bank Marketing Classification

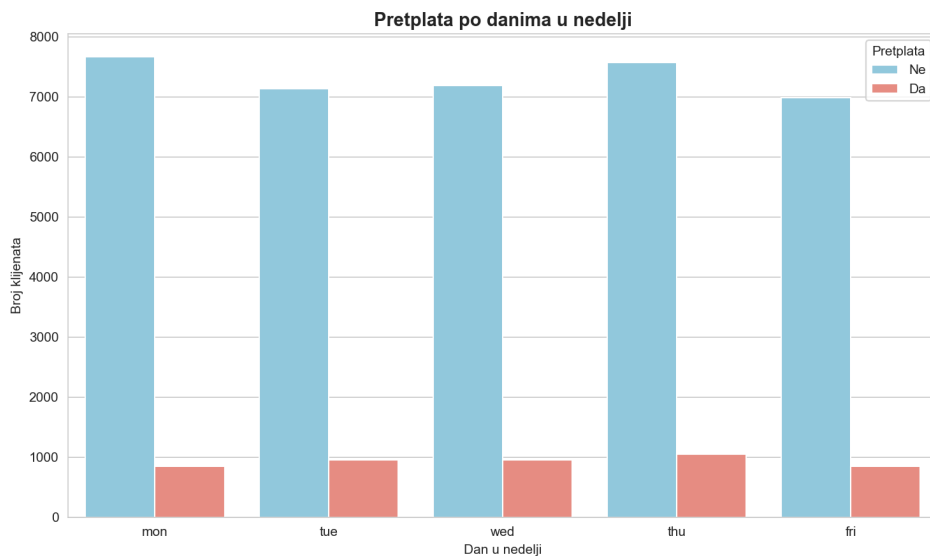
1. Opis problema

Cilj projekta je model koji predviđa da li će se klijent pretplatiti na bankovni rok-depozit (ciljna promenljiva y: yes/no) na osnovu demografskih, profesionalnih i marketing karakteristika. Problem je resen kao zadatak binarne klasifikacije, pri čemu je testirano više različitih algoritama uz optimizaciju njihovih hiperparametara. Zadatku je pristupljeno sa pretpostavkom da resursna ograničenja (poput cene i broja telefonskih poziva ili angazovanosti agenta) nisu primarni faktor optimizacije. Daleko je važnije identifikovati i privuci klijente koji su zaista spremni na pretplatu.

2. Analiza i priprema podataka

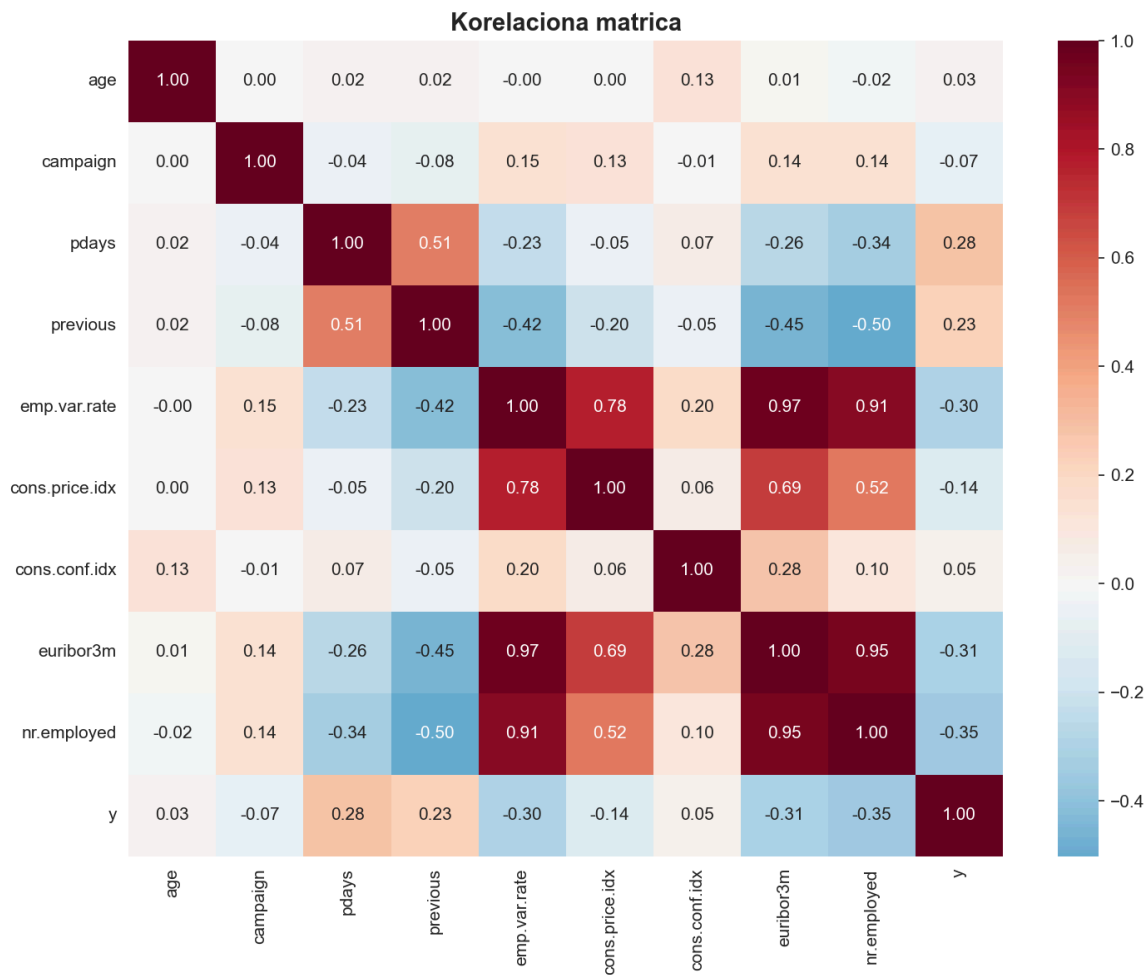
Celokupan proces obrade podataka podeljen je u dve celine: eksplorativna analiza podataka je implementirana unutar skripte **EDA.py**, a finalna priprema podataka u skripti **preprocessing.py**.

- Atribut “*duration*” (trajanje poziva) je irelevantan podatak za analizu, jer nije dostupan za vreme razgovora i smatra se nedostupnim u trenutku donosenja odluke.
- Takođe, atribut “*day_of_week*” (dan u nedelji kada je obavljen poziv) se ne smatra toliko značajnim za konačnu odluku. Kao što se sa grafika vidi raspodela “da” i “ne” je gotovo identična kroz sve dane u nedelji.



Pretplata po danima u nedelji

- Ekonomski atributi “*emp.var.rate*”, “*euribor3m*” i “*nr.employed*” su visoko korelisani i prenose istu informaciju. Pa je konacna odluka da se 2 od 3 izbace -> ostavljen je “*nr.employed*” kao njihov predstavnik.



Koleraciona matrica

- Atribut “*pdays*” (broj dana koji su prošli od poslednjeg kontakta u prosjoj kampanji) je izmenjen, tako da umesto vrednosti 999 prikazuje -1 (za osobu koja ranije nije bila kontaktirana).
- Primecena je pojava znacajnog broja “unknown” vrednosti, koje su ostavljene kao validan podatak i enkodirane zajedno sa ostatkom. Njihovim uklanjanjem izbacili bismo oko 25% podataka. Npr: najveći broj “unknown” vrednosti se pojavljuje kod atributa “*default*”, sto je logično jer klijenti nerado dele informacije o kreditnim zaduženjima.

job	330
marital	80
education	1730
default	8596
housing	990
loan	990
contact	0
poutcome	0
month	0

Broj "unknown" vrednosti po kategorijskim atributima

Dalje, odradjen je OneHotEncoder za sve kategorijske attribute i skaliranje numerickih podataka kako bi se sveli na zajednicki opseg. Koriscen je StandardScaler (mean = 0, std =1) kako bi se sprecilo da atributi sa vecim opsegom vrednosti dominiraju modelima osetljivim na skalu (KNN, logisticka regresija).

Podaci su podeljeni u odnosu 80% trening i 20% test skup, uz ocuvanje distribucije ciljne promenljive (stratify=y).

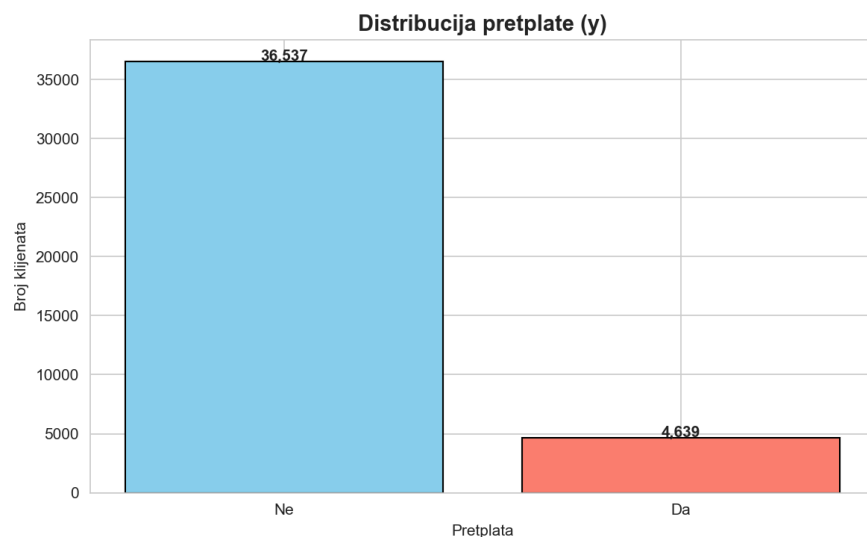
```
Pre enkodiranja:
X_train: (32940, 16)
X_test: (8236, 16)
```

```
Train shape nakon preprocessing-a: (32940, 46)
Test shape nakon preprocessing-a: (8236, 46)
```

Primetno povecanje broja kolona nakon enkodiranja (sa 16 na 46) i broj atribura u train/test skupu pri podeli 80/20

Napomena: originalno je bilo 20 kolona, ali su izbacene gore navedene

Zbog izrazenog disbalansa ciljne promenljive y (89%/11%) primenjena je funkcija SMOTE, koja vestacki generise nove podatke u dataset-u kako bi izjednacila odnos yes/no. *Napomena: kasnije je primenjena SMOTE funkcija unutar treninga (dok je ovde zanemarena), kako ne bi doslo do curenja podataka usled K-fold unaksne validacije.*



Odnos yes/no za ciljanu promenjivu (originalan set podataka)

Pre SMOTE: (32940, 46), DA=3711, NE=29229

Posle SMOTE: (58458, 46), DA=29229, NE=29229

Odnos yes/no nakon primene SMOTE funkcije (samo za train skup)

ANOMALIJE?

U skripti **preprocessing.py** je razmatrano uklanjanje anomalija upotrebom algoritma Isolation forest, ali nije primenjeno. Daljom analizom zakljuceno je, da vrednosti numerickih atributa nemaju ekstremnih raspona kako bi uklazivale na prave anomalije, osim godina klijenata - ali ne postoji ogranicenje vezano za godine (testirano na <18 i >90 -> izlaz ciljne promenjive moze biti i yes i no). Zanimljiv je zakljucak da je stopa preplate znatno veca za lica +60 godina.

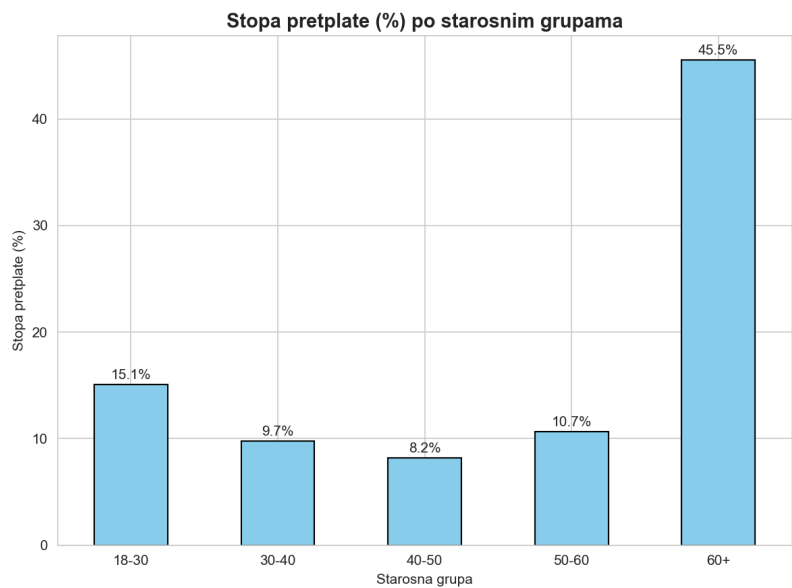
Prestari:

27826	95	no
38022	91	no
38032	91	no
38452	98	yes
38455	98	yes
38921	94	no
39655	92	no
39734	92	yes
40450	92	yes
40469	92	yes

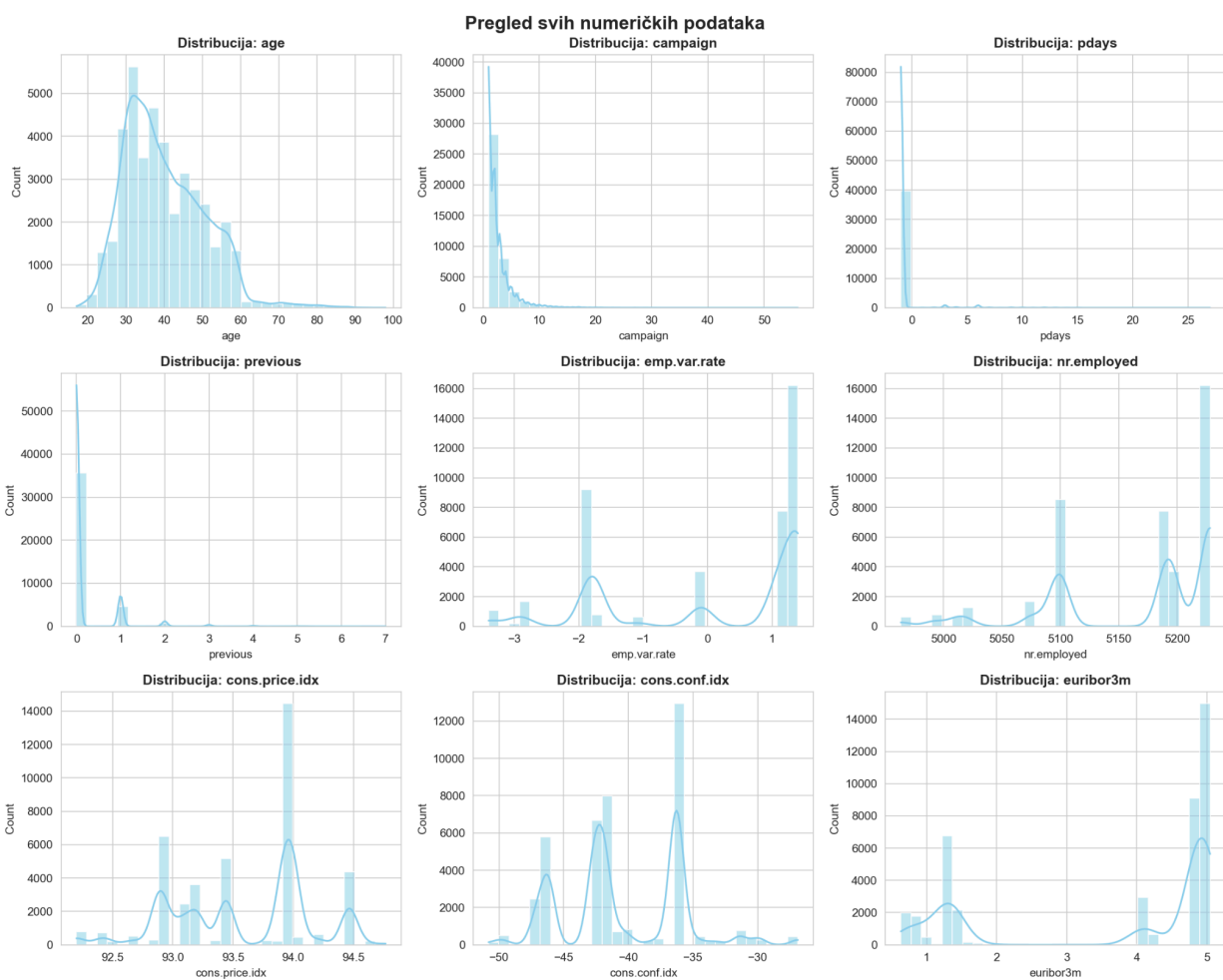
Maloletni:

37140	17	no
37539	17	no
37558	17	no
37579	17	yes
38274	17	yes

Raspodela ciljne promenjive y za granicne godine klijenata



Stopa preplate po godinama klijenata



Raspon i raspodela vrednosti brojnih atributa

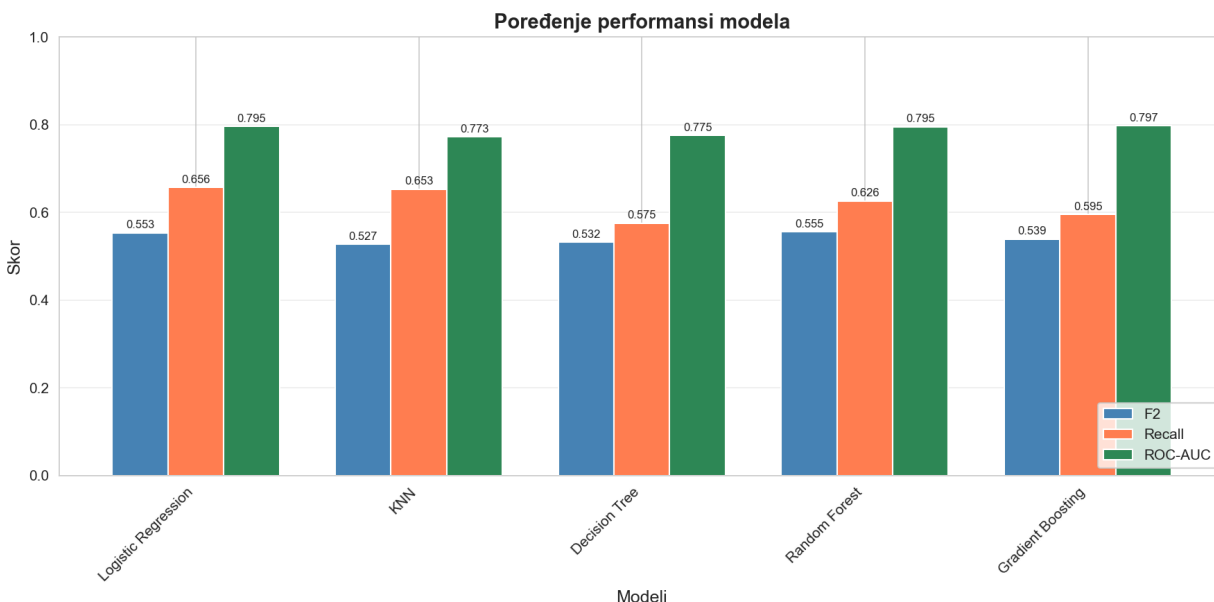
3. Odabir i treniranje modela

Svi modeli su trenirani u skripti **train.py**. Primenjen je GridSearch za izbor hiperparametara, gde K-fold unakrsna validacija (k=5) preuzima ulogu val skupa za njihov izbor. U skladu sa pretpostavkom da resursna ogranicenja nisu primerni faktor, hiperparametri su optimizovani pomocu F2 score metrike kako bi se osigurala stabilnost modela i sprecilo pronalazenje trivijalnih resenja sa izrazito niskom preciznoscu. F2 score racuna harmonijsku sredinu (slicno kao i F1 score), ali vise naglasava uticaj recall-a (u nasem slucaju duplo vise). Racuna se odnos resursa koje imamo na raspolaganju i broj onih koje smo uspesno pozvali. Time obezbedujemo da model primarno juri recall, ali zadržavamo kaznu za preveliki broj laznih uzbuna. Jednom kada imamo stabilan model koji uspesno balansira bias i varijansu, ciljani recall mozemo dodatno podizati spustanjem praga odlucivanja ispod default-nog (0,5). Recall metrika nam je veoma znacajna, jer ne zelimo da propustimo potencijalnog klijenta koji bi se pretplatio.

Napomena: u kodu je deo sa izborom hiperparametara zakomentaran, jer sam vise puta pokretala tu skritu iz razlicitih razloga. Oni hiperparametri koji su posavljani po modelima su zapravo izabrani GridSearch-om.

Model	F2	Recall	Precision	F1	ROC-AUC
Logistic Regression	0.5532	0.6562	0.3398	0.4478	0.7954
KNN	0.5266	0.6530	0.2968	0.4081	0.7725
Decision Tree	0.5316	0.5754	0.4073	0.4770	0.7748
Random Forest	0.5552	0.6261	0.3822	0.4747	0.7951
Gradient Boosting	0.5389	0.5948	0.3915	0.4722	0.7971

Poredjenje metrika svih modela sa default pragom (0.5)



Poređenje prerformansi modela

Na osnovu testiranih algoritama, izdvojena su dva modela kao najrelevantnija za dati problem: Logisticka regresija i Random Forest.

RANG LISTA (prema F2 skor):

- ```

1. Random Forest: F2=0.5552, Recall=0.6261
2. Logistic Regression: F2=0.5532, Recall=0.6562
3. Gradient Boosting: F2=0.5389, Recall=0.5948
4. Decision Tree: F2=0.5316, Recall=0.5754
5. KNN: F2=0.5266, Recall=0.6530
```

*Rang lista najboljih modela prema F2 skor i default-nim pragom*

### 3.1 Baseline model - Stratified Dummy Classifier

Stratified Dummy Classifier je model koji nasumicno predvidja klase yes/no, postujuci distribuciju trening skupa ("no" 89% i "yes" 11%). Ovaj baseline model predstavlja performanse koje bismo postigli bez ikakvog modela. Svi testirani modeli su bolji od ovog modela. Implementacija i poredjenja baseline modela se nalaze u skripti ***analysis.py***.

✚ Stratified baseline (nasumično predviđanje):

Accuracy: 0.8066  
Precision: 0.1350  
Recall: 0.1325 (13.3%)  
F1: 0.1338  
F2: 0.1330  
ROC-AUC: 0.5124  
Predviđeno DA: 11.1%

*Metrike baseline modela*

 1. NAJBOLJI PO F2 SKORU: Random Forest

| Metrika   | Baseline | Model  | Razlika |
|-----------|----------|--------|---------|
| F1        | 0.1338   | 0.4747 | +0.3409 |
| F2        | 0.1330   | 0.5552 | +0.4222 |
| Recall    | 0.1325   | 0.6261 | +0.4935 |
| Precision | 0.1350   | 0.3822 | +0.2472 |

*Poredjenje baseline modela sa Random Forest-om*

 2. NAJBOLJI PO RECALL-U: Logistic Regression

| Metrika   | Baseline | Model  | Razlika |
|-----------|----------|--------|---------|
| F1        | 0.1338   | 0.4478 | +0.3140 |
| F2        | 0.1330   | 0.5532 | +0.4202 |
| Recall    | 0.1325   | 0.6562 | +0.5237 |
| Precision | 0.1350   | 0.3398 | +0.2048 |

*Poredjenje baseline modela sa Logisickom regresijom*

## 3.2 Logistic Regression

Logisticka regresija ostvaruje najveći recall među svim testiranim modelima - linearna granica odlučivanja uspe da izoluje 65,6% od svih klijenata. Uspesno identifikuje skoro 2/3 klijenata koji bi uplatili depozit.

Koriscena je Lasso (L1) regularizacija koja predvidja koji atribut ne pomaze modelu, tj obezbedjena je automatska selekcija atributa (svodjenje manje bitnih koeficijenata na nulu). Cime je varijansa (osetljivost na konkretan train skup) svedena na minimum, a generalizacija (balans izmedju tacnosti na train podacima i jednostavnosti modela) svedena na maksimum.

### 3.3 Random forest

Ostvaruje stabilan i snazan recall od 62,61%, ali bolje kontrolise lazne izbune (preciznost = 38,22%). To doprinosi i solidnom F2 scoru od 55,52%.

Kao rezultat dobijemo model koji odlicno generalizuje i predstavlja najstabilnije resenje jer uspesno balansira izmedju bias-a (koliko je model pojednostavio problem) i varijanse. Glavni cilj Random forest algoritma je smanjenje varijanse, sto i uspevamo treniranjem 50 nezavisnih stabala.

### 3.4 KNN

Iako KNN belezi solidan recall (~65%), u praksi ovaj model nije preporucljiv za bankarski sistem, jer nakon OneHotEncoder-a prostor atributa se znacajno povecava, a mana ovog algoritma je da daje slabije rezultate u prostoru visoke dimenzionalnosti. Takodje upotrebom SMOTE funkcije, povecava nam se baza podataka, cime se povecava vreme potrebno za trening (jer mora racunati rastojanje do svakog).

### 3.5 Decision Tree

Najlosiji rezultat na recall-u (57,5%) cime se automatski eliminise iz uzeg izbora. Pored toga ima sklonost za overfittitng i visoku varijansu (bez obzira na primenu SMOTE-a, stablo ostaje bez predstavnika manjinske klase i dobijamo losije rezultate).

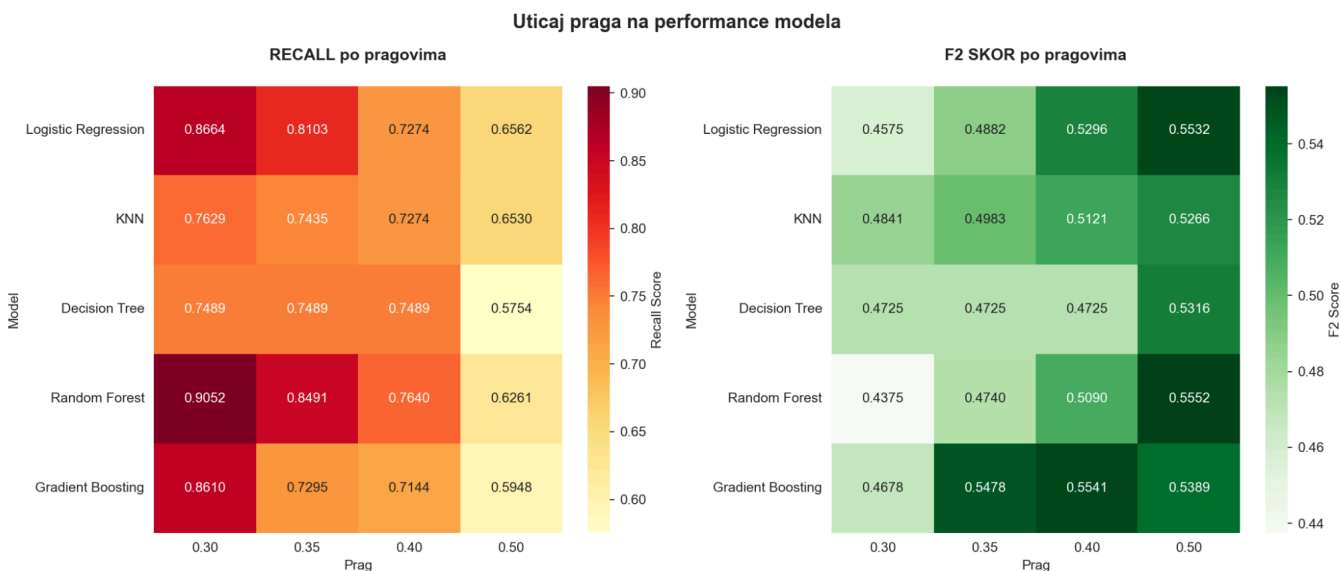
### 3.6 Gradient Boosting

Gradient Boosting pokazuje najveći potencijal pri snizavanju praga odlucivanja - na pragu 0,4 postiže  $F2=0.5541$ , cime postaje najstabilniji model. *Napomena: ovaj model nije izabran za deployment, vec samo najbolji trenirani modeli na default-nom pragu odlucivanja (0,5).*

### 3.7 Uticaj praga odlucivanja

Testiranje praga odlucivanje nalazi se u skripti ***analysis.py***.

Podrazumevani klasifikacioni prag odlucivanja vecine algoritama masinskog ucenja iznosi 0,5. Ukoliko je verovatnoca da klijent pripada klasi "da" veca od 50% model ga klasifikuje kao pozitivnog. Posto je primarni cilj banke da maksimizuje broj klijenata koji bi se pretplatili (visok recall), sprovedeno je testiranje sa smanjenim pragom odlucivanja. Snizenjem praga, banka porucuje modelu da pozove klijenta i ako si samo 30%/40% sigurna da ce uplatiti depozit.



*Uticaj snizenih pragova na recall (levo) i F2 skor (desno) na svim modelima*

**Zakljucak:**

**3.7.a.**

Najbolji recall dobijemo sa modelom Random Forest, ako smanjimo prag odlucivanja na 0,3.

Time veoma zrtvujemo preciznost, te nam F2 skor pada ispod 0,5.

Slicno se desi i sa Logistickom regresijom na pragu 0,3.

**3.7.b.**

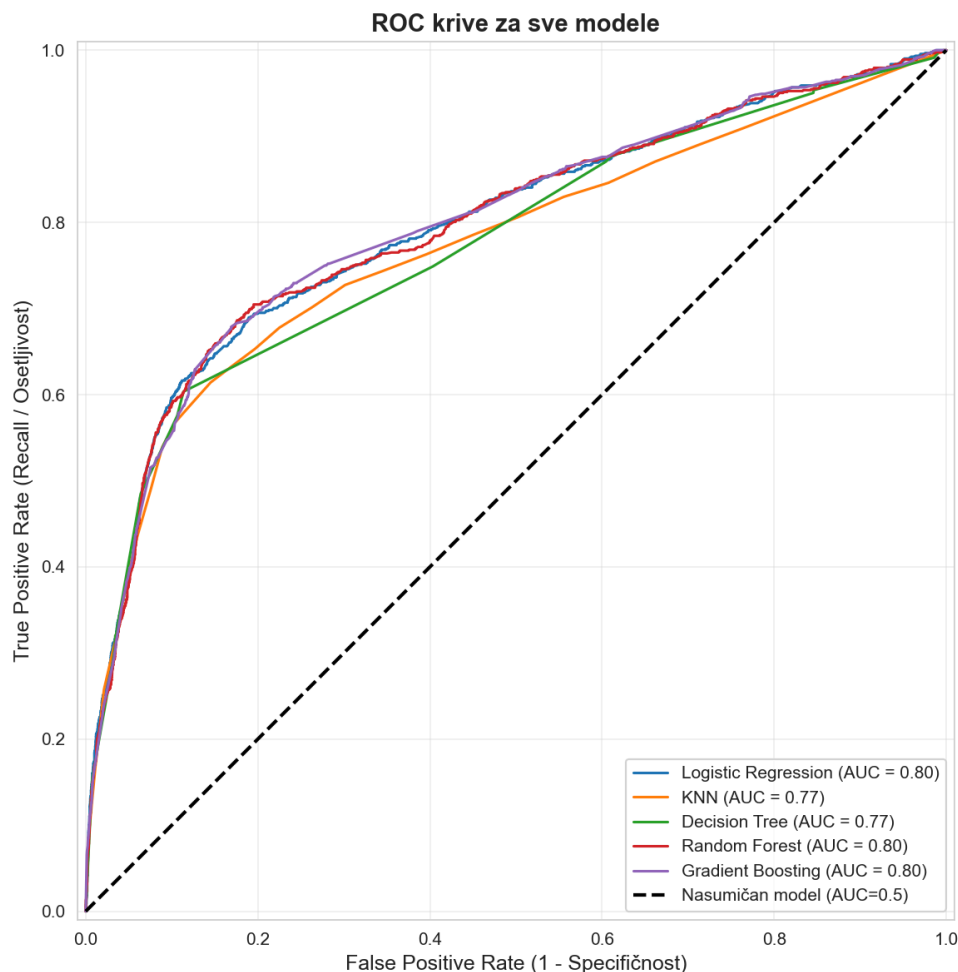
Zanimljiv podatak je da Gradient Boosting ima bolji F2 skor sa snizenim pragom odlucivanja

(0,4), cime se ukljucuje u trku izbora najboljeg modela. Cak i smanjenjem praga na 0,35

dobijamo bolje vrednosti obe metrike, nego sa default-nim pragom. Za razliku od ostalih modela

kod kojih smanjenje praga dovodi do naglog kolapsa preciznosti, sekvencijalna priroda učenja

Gradient Boosting-a omogucava drastican porast recall-a uz zadrzavanje stabilne preciznosti.



*ROC - kriva koja prikazuje ponasanje modela kroz razlicite pragove odlucivanja*

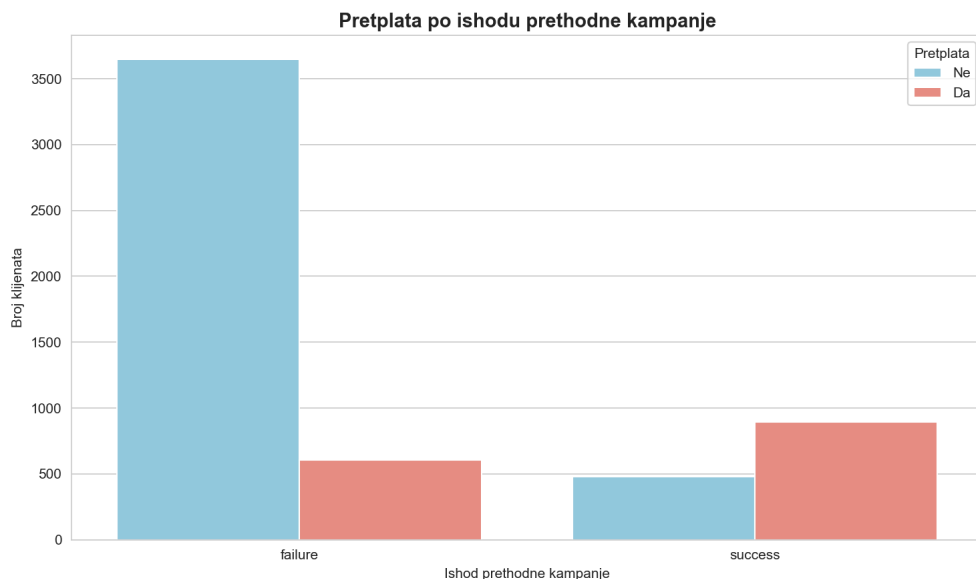
Gornji desni ugao (prag = 0) - za sve predpostavimo da ce se pretplatiti (veci recall, ali i vise lazno pozitivnih slucajeva).

Donji desni ugao (prag = 1) - za sve predpostavimo da se nece pretplatiti (nizak recall, nemamo lazno pozitivne).

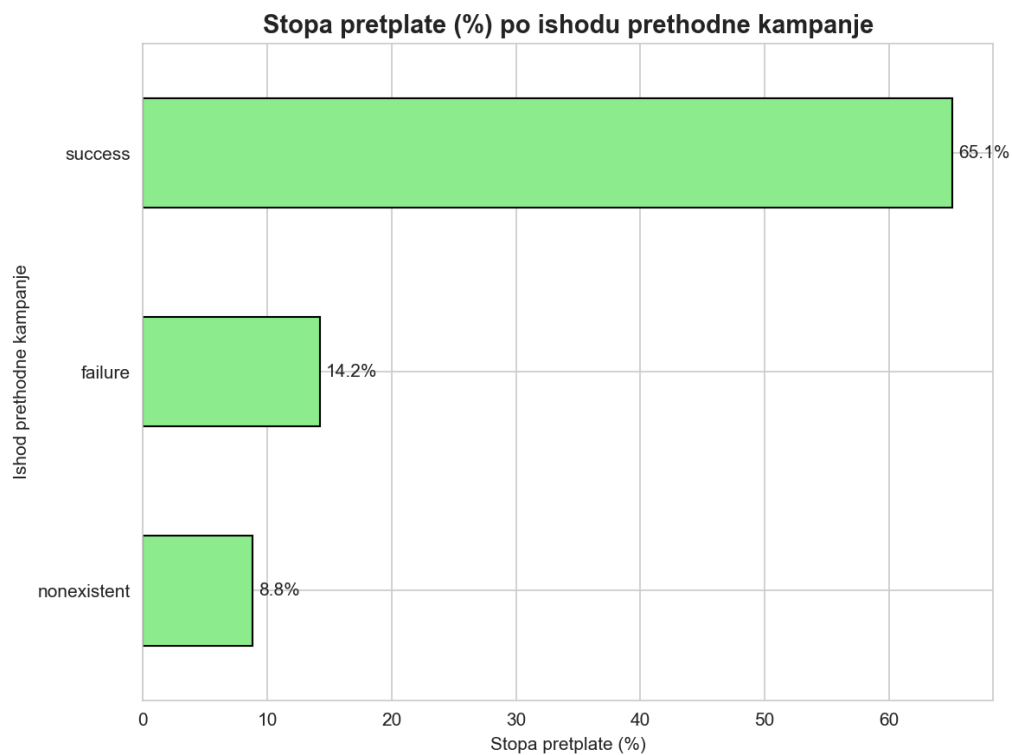
## 4. Odabir najznacajnijih atributa

Analiza atributa izvedena je u skripti **feature.py** gde su ispitivana tri modela (Logisticka regresija, Random Forest i Gradient Boosting).

- Ekonomski indikatori dominiraju kod sva tri modela, sto ukazuje na to da spremnost klijenta na stednju direktno zavisi od opsteg stanja u ekonomiji.
- Efekat prethodne kampanje (atribut "poutcome") je najvazniji za Logisticku regresiju. Klijenti koji su ranije imali pozitivno iskustvo sa bankom (ucestvovali su u prethodnoj kampanji) imaju vecu sansu da ponovo prihvate ponudu.

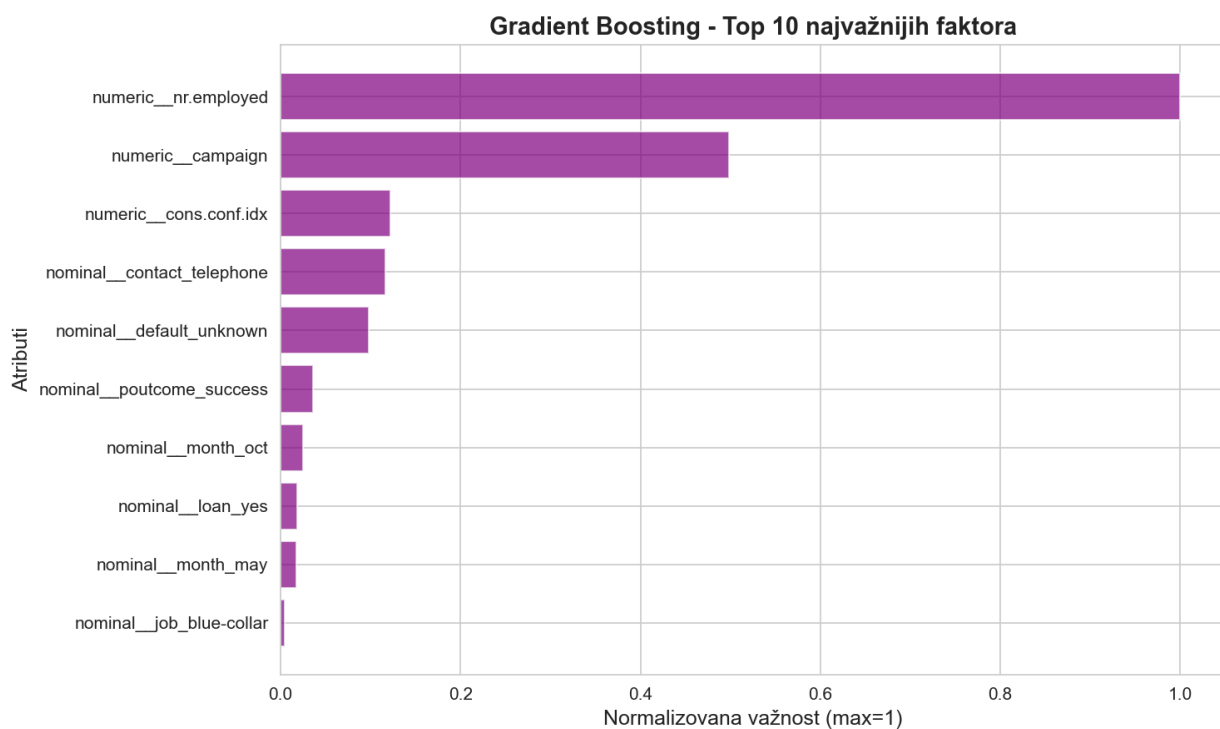


*Pretplata po ishodu prethodne kampanje: failure - nije prihvatio prethodnu kampanju i success - prethodno je prihvatio kampanju*

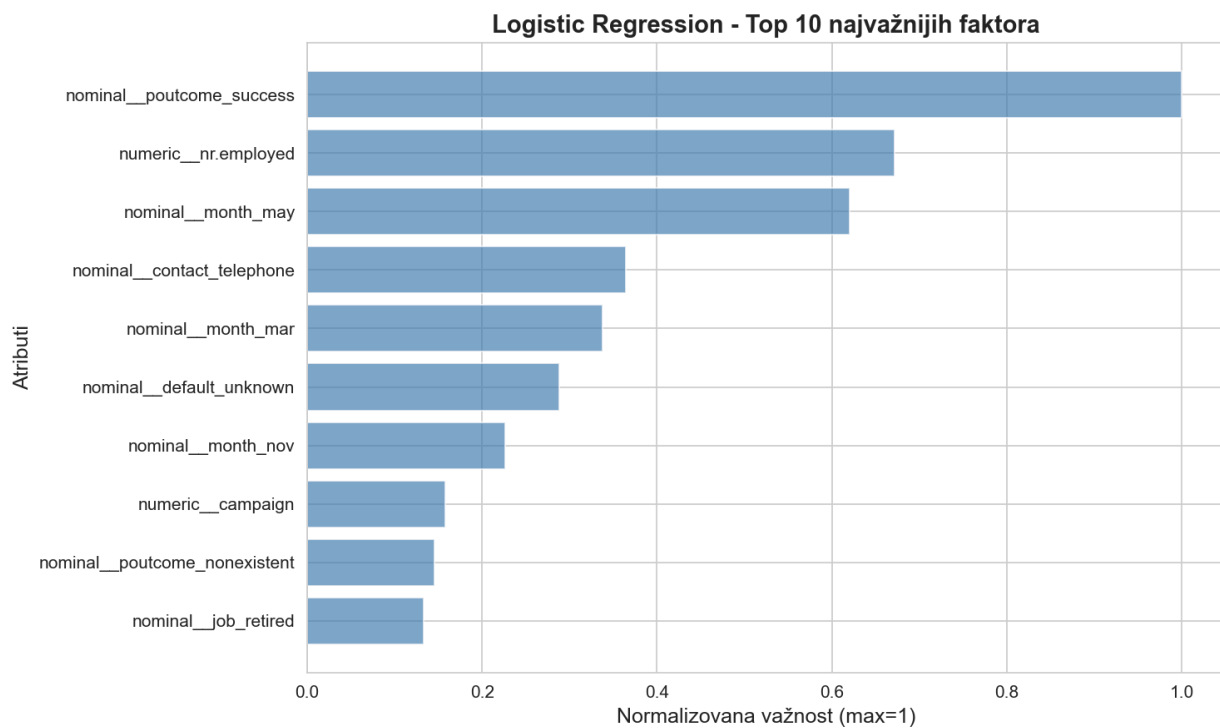


*Stopa pretplate po ishodu prethodne kampanje (success - prethodna kampanja uspesna, failure - prethodna kampanja bezuspesna i nonexistent - nije bio kontaktiran u prethodnoj kampanji)*

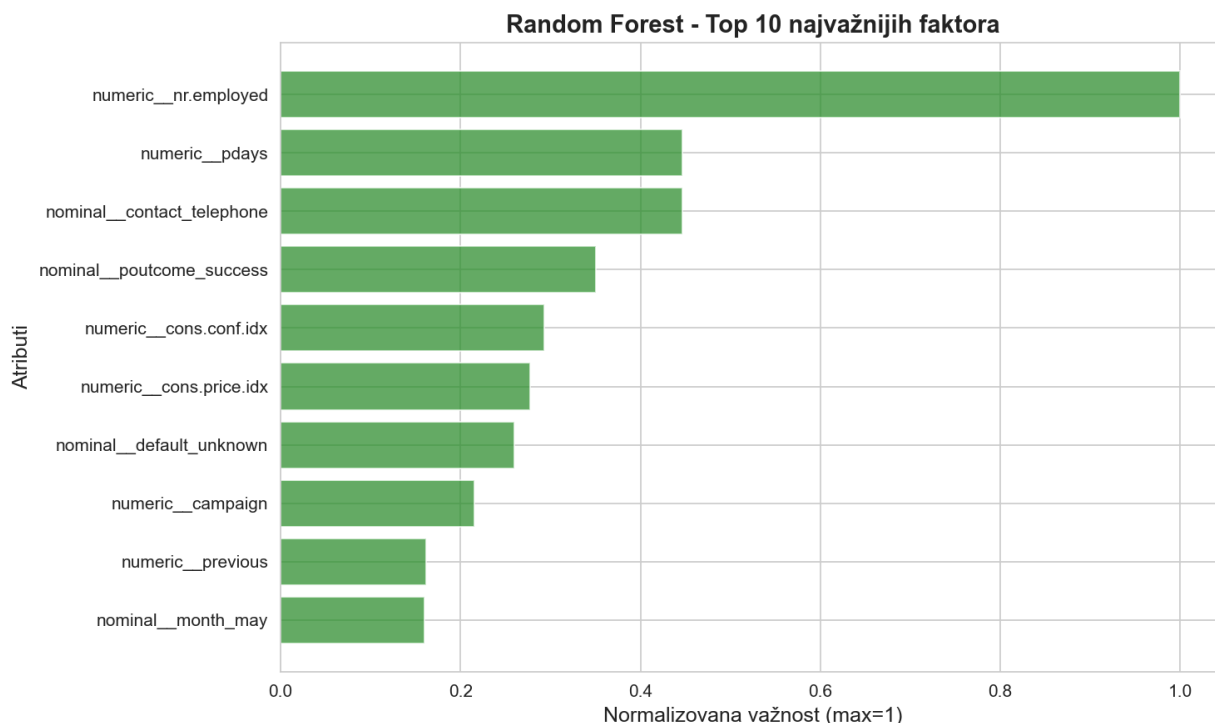
- Takodje, operative karakteristike poziva: broj kontakata tokom tekuće kampanje ("campaign") i tip komunikacije ("contact\_telephone") se konstantno pojavljuju u top 10.



*Top 10 najznacajnijih atributa za Gradient Boosting*



*Top 10 najznacajnijih atributa za Logisticku regresiju*



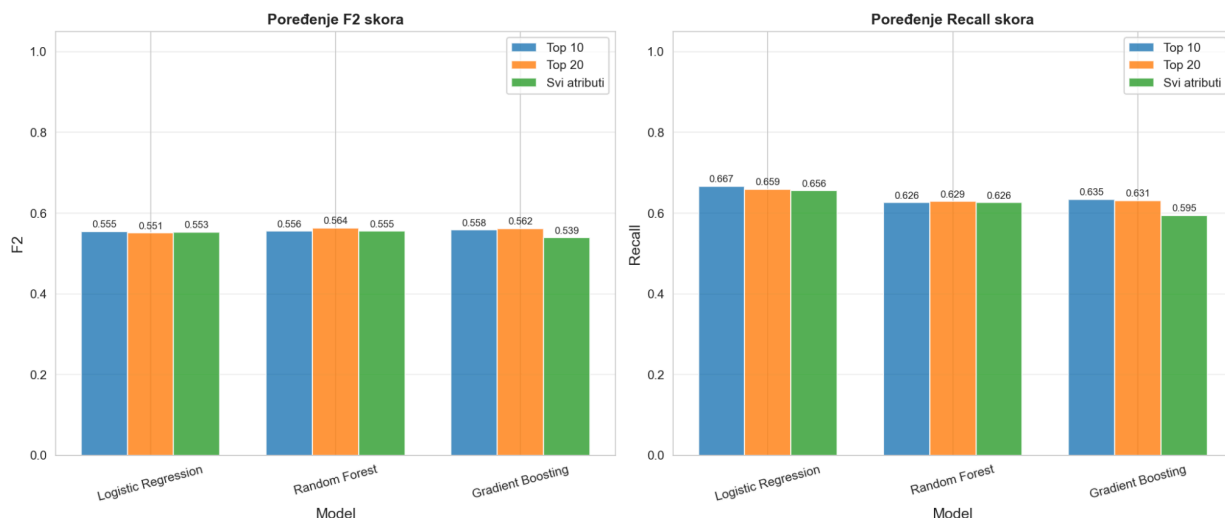
*Top 10 najznacajnijih atributa za Random Forest*

### **Zakljucak:**

Smanjenje broja atributa ne narušava performanse ni kod jednog od testiranih modela, nemamo značajnu razliku u rezultatima ni na top 10, ni top 20 najznacajnijih atributa. Koriscenjem top 20 atributa dobijamo racunski znatno lakse i brze modele , a zadržavamo diskriminatorsnu moc.

Logisticka regresija postize najbolji recall (~67%) sa samo 10 najznacajnijih atributa, unosenje preostalih unosi blagi sum (regularizacija je dobro eliminisala nebitne attribute). Time doprinosimo poboljsanju ako zelimo da maksimizairamo recall.

Gradient Boosting je veoma stabilan i performanse sa 20 atributa i svih 46 su potpuno identicne, sto znaci da je algoritam kroz stablo odlucianja ignorisao nebitne attribute. Random Forest ima najbolji F2 skor sa top 20 atributa, ali neznatno od punog skupa. Razlika je mala, ali sugerise da neki od 46 attribute unose sum -> zato bi pre izabrali ovaj model, nego pocetni.



*Poredjenje tri ispitana algoritma sa top 10 vs top 20 vs svim znacajnim atributima (F2 score - levo i recall - desno)*

## 5. Deployment modela

Modeli su eksportovani kao .pkl fajlovi pomocu joblib biblioteke i dostupni su u Streamlit-u UI (**ui.py**) koji:

- učitava preprocessor i oba modela pri pokretanju
- korisnik bira strategiju (max\_f2 ili max\_recall) u zavisnosti od poslovnog cilja
- unosi vrednosti atributa kroz formu
- klik na Predict vraća: predikciju (yes/no), verovatnocu pretplate i vizuelni progress bar

Eksportovani modeli:

- best\_f2\_model.pkl — Random Forest, optimizovan po F2 skoru
- best\_recall\_model.pkl — Logisticka regresija, optimizovana po Recall-u
- preprocessor.pkl — OneHotEncoder + StandardScaler, isti koji je koriscen pri treningu

Skripta se pokrece: `streamlit run ui.py`

### Strategije odlucivanja:

Banka bira jednu od dve strategije u zavisnosti od poslovnog konteksta:

1. max\_recall (Logisticka regresija, Recall=65.6%) — kada banka ima dovoljno resursa i ne sme propustiti nijednog potencijalnog klijenta. Model ce pozvati vise ljudi, ali ce uhvatiti najveći broj stvarnih pretplatnika.

2. max\_f2 (Random Forest, F2=55.5%) — kada postoje ogranicenja u broju poziva ili angazovanih agenata. Model pravi manji broj lazni uzbuja uz zadrzavanje solidnog recall-a.

## 6. Zakljucak

Projekat je pokazao da je moze izgraditi solidan model za predvidjanje pretplate klijenata na depozit, ali i dalje kljucni problem lezi u disbalansu klasa (89%/11%) i u 25% “unknown” vrednosti. Faktori koji su to ublazili su primena SMOTE funkcije unutar K-fold unakrsne validacije i odabir F2 metrike (umesto F1) koja favorizuje recall.

Od pet testiranih algoritama, izdvojena su dva kao najrelevantnija: Logisticka regresija za scenarije gdje je prioritet maksimalan recall, i Random Forest za scenarije gde je potreban balans izmedju recalla i preciznosti. Takodje, ima dosta prostora za napredak, dodatna obrada “unknown” vrednosti i detaljno podesavanje praga odlucivanja.

Analiza najznacajnijih atributa pokazala je da ekonomski indikatori i ishod prethodne kampanje dominantno odredjuju verovatnocu pretplate. Dodatno, smanjenje broja atributa sa 46 na 20 ne narusava performanse, sto otvara mogucnost za brze i jednostavnije modele u produkciji